UNIVERSIDADE ESTADUAL PAULISTA "JÚLIO DE MESQUITA FILHO"
CAMPUS DE SOROCABA

CURSO DE ENGENHARIA DE CONTROLE E AUTOMAÇÃO

VITOR CORREIA MENDES

Trabalho Individual II – SCOM

Backend, persistência e segurança: *Cuphead Boss Guide*

SOROCABA
2026

Índice

1 Introdução **3**

2 Evolução em relação ao Trabalho 1 **3**

3 Requisitos funcionais **5**

4 Arquitetura do sistema **6**
4.1 Organização dos diretórios 6
4.2 Injeção de dependências 7
4.3 Fluxo de uma requisição 7

5 Modelo de dados **7**

6 Estratégia de autenticação **9**
6.1 Controle de acesso e matriz de permissões 9

7 Estratégia de segurança **10**
7.1 Validação e sanitização 11
7.2 Tratamento de erros 12

8 Endpoints implementados **12**

9 Estratégia de testes **14**

10 Resultados dos testes de segurança **15**

11 Logs e observabilidade **19**

12 Desempenho e limitações **20**
12.1 Medição de desempenho 20
12.2 Limitações 20

13 Uso de IA generativa **21**

14 Conclusões **22**

Anexos **23**

A Script de criação do banco **23**

B Documentação da API **25**

C Resultado dos testes automatizados **34**

D Capturas da interface **36**

E Amostra dos logs **41**

F Histórico Git **42**

G Checklist de requisitos **43**

1 Introdução

Este relatório documenta a segunda etapa do projeto *Cuphead Boss Guide*, desenvolvido para a disciplina SCOM. No Trabalho Individual I foi construída uma interface web estática (HTML5 e CSS3) com estratégias para os chefes do jogo *Cuphead* e um checklist de conquistas. O Trabalho II evolui esse sistema para uma aplicação cliente-servidor completa, acrescentando backend funcional, persistência em banco de dados relacional, autenticação, controle de acesso por perfis, validação de entradas, proteções de segurança, testes automatizados, registro de logs e documentação da API.

A solução foi implementada com Node.js e Express no backend (*JavaScript everywhere*, como sugerido no enunciado) e PostgreSQL como banco de dados. A interface do Trabalho I foi preservada visualmente e passou a consumir a API por meio de JavaScript em módulos nativos, sem frameworks.

Tabela 1: Resumo da solução

Aspecto	Escolha
Frontend	HTML5, CSS3 e JavaScript (módulos ES), sem frameworks
Backend	Node.js 22 + Express 4, arquitetura em camadas
Banco de dados	PostgreSQL (testado na versão 16), driver `pg`
Autenticação	Sessão no servidor, bcrypt e token CSRF
Testes	`node:test` + supertest: 56 testes automatizados
Repositório	`https://github.com/akavirto123/scom-projeto-cuphead`

2 Evolução em relação ao Trabalho 1

O Trabalho I apontava, em sua seção de limitações, que o formulário "Sugerir Nova Tática" era apenas demonstrativo e que o checklist de conquistas não guardava o progresso do usuário. Ambas as limitações foram resolvidas. A Tabela 2 resume a evolução.

Tabela 2: Comparação entre o Trabalho I e o Trabalho II

Item	Trabalho I (`v1.0`)	Trabalho II (`v2.0`)
Chefes	19 cartões fixos no HTML	Tabela no banco; listagem paginada, filtro por ilha e busca; CRUD pelo administrador
Formulário de tática	Sem efeito (`action="#"`)	Grava a tática como pendente; moderação pelo administrador; táticas aprovadas aparecem no card do chefe
Conquistas	Lista estática	Catálogo no banco e progresso salvo por usuário, com barra de percentual
Usuários	Inexistentes	Cadastro, login, logout e três perfis (visitante, jogador, administrador)
Páginas	`index.html`	`index.html`, `conta.html` e `admin.html`
Organização	Dois arquivos na raiz	`frontend/`, `backend/` e `docs/`
Qualidade	Lighthouse e W3C	Testes automatizados, testes de segurança, logs e medição de desempenho

A identidade visual (paleta “Vermelho Cuphead” e “Amarelo Vintage”, fundo de pergaminho, fontes Bangers e Outfit) foi mantida, e os novos componentes (formulários de login, tabelas do painel, selos de status) foram estilizados com as mesmas variáveis CSS do Trabalho I. Duas mudanças de layout foram feitas: o cabeçalho fixo passou a valer apenas a partir de 768 px, porque no celular, com a nova área de login, ele ocupava quase metade da tela; e os cartões de chefes passaram a ser paginados (6 por página) em vez de exibidos todos de uma vez.



Figura 1: Página inicial do Trabalho II: filtros, cartões vindos da API e checklist de conquistas

3 Requisitos funcionais

Tabela 3: Requisitos funcionais e perfis envolvidos

ID	Requisito	Perfil
RF01	Listar chefes com paginação, filtro por ilha e busca por nome ou fase	Todos
RF02	Exibir detalhes de um chefe e as táticas aprovadas da comunidade	Todos
RF03	Cadastrar conta (perfil jogador por padrão)	Visitante
RF04	Entrar e sair do sistema	Visitante / logado
RF05	Enviar tática para um chefe (nasce como pendente)	Jogador, admin
RF06	Ver, editar e excluir as próprias táticas enquanto pendentes	Jogador, admin
RF07	Marcar e desmarcar conquistas e ver o percentual concluído	Jogador, admin
RF08	Cadastrar, editar e excluir chefes	Admin
RF09	Moderar táticas: listar por status, aprovar, rejeitar e excluir	Admin

ID	Requisito	Perfil
RF10	Listar usuários e alterar o perfil de outro usuário	Admin
RF11	Consultar a saúde da API e do banco	Todos

Requisitos não funcionais atendidos: senhas armazenadas apenas como hash bcrypt; toda entrada validada no backend; respostas JSON padronizadas; registro de eventos com *timestamp*; configuração por variáveis de ambiente; execução local com três comandos (`npm install`, `npm run db:migrar`, `npm start`).

4 Arquitetura do sistema

Foi adotada uma arquitetura cliente-servidor com o backend organizado em **camadas**. A "visão" é o frontend estático, que consome a API JSON; o backend é dividido em camada HTTP (rotas e controllers), camada de lógica de aplicação (services, regras de domínio e validação) e camada de acesso a dados (repositories). Cada camada depende apenas da camada imediatamente abaixo: controllers não escrevem SQL, repositories não conhecem objetos de requisição e services não sabem que existe HTTP.

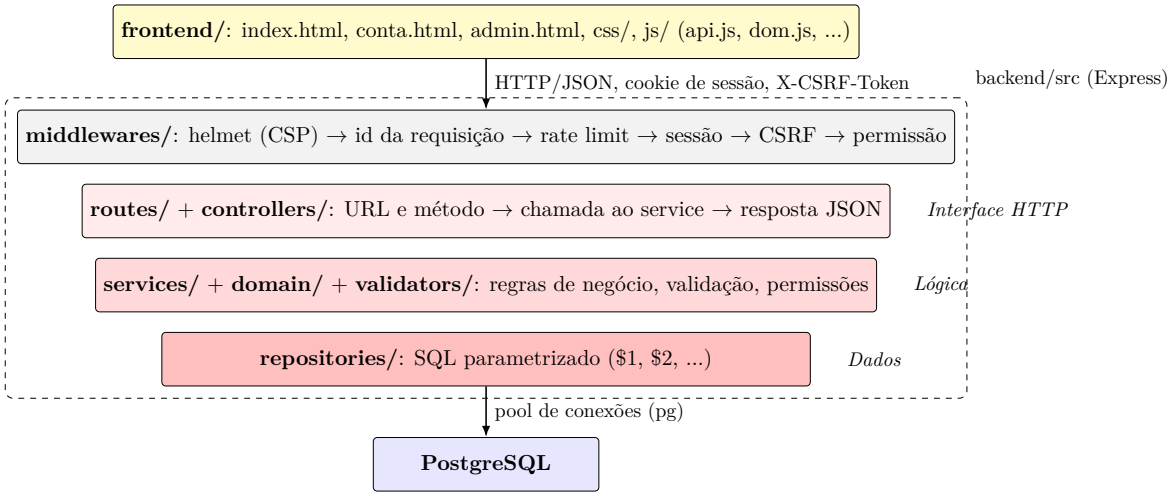


Figura 2: Camadas da aplicação

4.1 Organização dos diretórios

```
scom-projeto-cuphead/
|-- frontend/            interface (arquivos estáticos)
|   |-- index.html  conta.html  admin.html
|   |-- css/style.css
|   `-- js/              api.js, dom.js, cabecalho.js, inicio.js, conta.js, admin.js
|-- backend/
|   |-- src/
|   |   |-- server.js    ponto de entrada
|   |   |-- app.js       composition root: liga as camadas
```

```
|   |   |-- config/      env.js, database.js, logger.js
|   |   |-- middlewares/ sessao, csrf, autorizacao, limitadores, tratadorErros
|   |   |-- routes/  controllers/  services/  repositories/
|   |   |-- validators/  esquemas de entrada
|   |   |-- domain/      permissoes.js (matriz RBAC)
|   |   `-- utils/       AppError, validacao, http
|   |-- database/        migrations/, script_completo.sql, executor
|   |-- scripts/         criarAdmin.js, benchmark.js, demonstrar-seguranca.sh
|   |-- tests/           unit/ e integracao/
|   `-- .env.example
`-- docs/                API.md, ARQUITETURA.md, CHECKLIST.md, evidencias/, relatorio/
```

4.2 Injeção de dependências

Cada módulo exporta uma *fábrica*, por exemplo `criarChefeService({ chefeRepository, ilhaRepository, logger })`. O arquivo `app.js` é o único ponto que instancia e conecta as camadas. Com isso, os testes unitários substituem os repositories por versões em memória e os testes de integração montam a aplicação real com outra configuração (banco de testes, *rate limit* menor e um logger que captura os eventos).

4.3 Fluxo de uma requisição

Tomando como exemplo um jogador enviando uma tática (`POST /api/taticas`): (1) o `helmet` aplica os cabeçalhos de segurança; (2) é gerado um `requisicaoId`; (3) o corpo JSON é lido com limite de 10 KB; (4) a sessão é carregada do PostgreSQL pelo cookie; (5) o usuário da sessão é buscado no banco; (6) o token CSRF do cabeçalho é comparado com o da sessão; (7) a matriz de permissões é consultada; (8) o controller chama `taticaService.criar`; (9) o service valida e sanitiza os dados, confere se o chefe existe, grava pelo repository e registra o evento `entidade.criada`; (10) qualquer exceção em qualquer etapa é convertida em resposta JSON pelo tratador central de erros.

5 Modelo de dados

O banco possui oito tabelas. A entidade principal é `chefes`; `taticas` e `usuario_conquistas` registram a participação dos usuários. A Figura 3 mostra os relacionamentos.

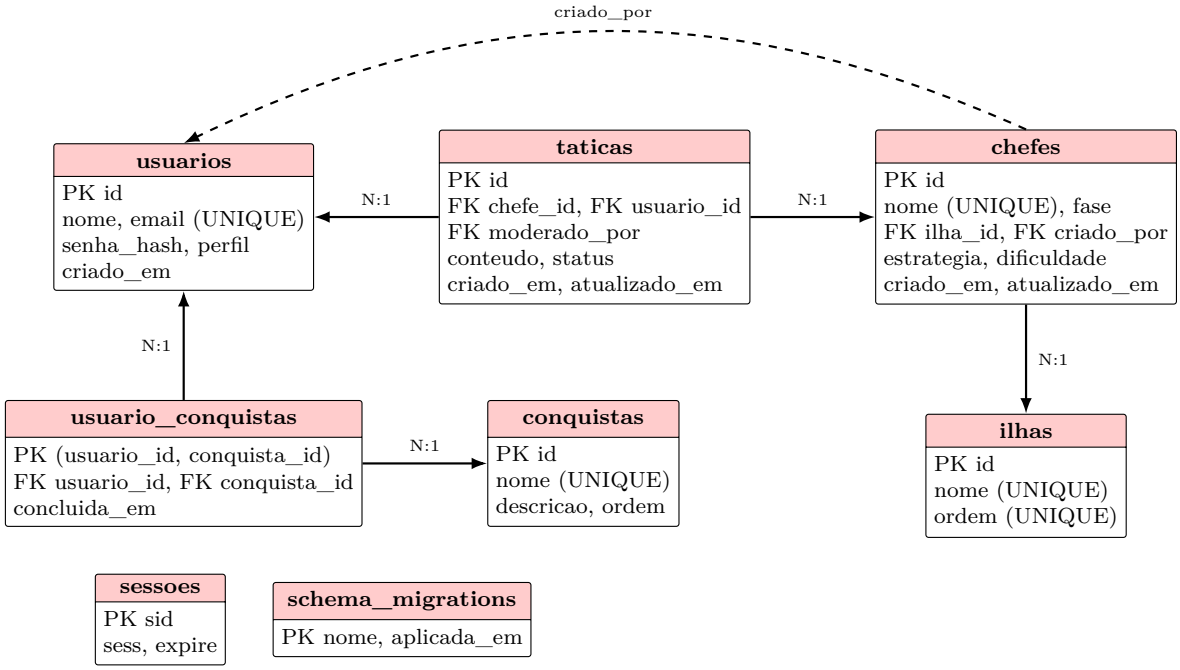


Figura 3: Modelo entidade-relacionamento (setas apontam para a tabela referenciada)

Tabela 4: Constraints e índices

Tipo	Aplicação
PRIMARY KEY	Todas as 8 tabelas (`usuario_conquistas` usa PK composta)
FOREIGN KEY	7 FKs: `ON DELETE CASCADE` para táticas e progresso, `RESTRICT` para ilha do chefe, `SET NULL` para autor/moderador
UNIQUE	e-mail do usuário, nome do chefe, nome e ordem da ilha, nome e ordem da conquista
NOT NULL	Todas as colunas obrigatórias
CHECK	perfil ∈ {jogador, admin}; e-mail em minúsculas; dificuldade 1–5; status ∈ {pendente, aprovada, rejeitada}; tamanhos de nome, estratégia e conteúdo
Índices	`chefes(ilha_id)`, `chefes(lower(nome))`, `taticas(chefe_id, status)`, `taticas(usuario_id)`, `taticas(status, criado_em)`, `sessoes(expire)`
Trigger	`definir_atualizado_em()` atualiza `atualizado_em` em chefes e táticas

Os tipos foram escolhidos pelo domínio: `SMALLINT` para ilhas, conquistas e dificuldade; `VARCHAR(n)` com o mesmo limite usado na validação; `TIMESTAMPTZ` para datas; chaves `GENERATED ALWAYS AS IDENTITY`.

Migrações

O esquema é versionado em `backend/database/migrations/` (`001_criar_tabelas.sql` e `002_dados_iniciais.sql`, que importa os 19 chefes e 28 conquistas do HTML do Trabalho I). O comando `npm run db:migrar` aplica apenas as migrações pendentes e

registra cada uma em `schema_migrations`. O arquivo `script_completo.sql`, gerado a partir das migrações por `npm run db:script`, é o script único de criação do banco e foi testado com `psql -v ON_ERROR_STOP=1` em um banco vazio (Anexo A).

6 Estratégia de autenticação

Foi escolhida autenticação por **sessão no servidor** em vez de JWT. Com sessão, o logout invalida o acesso imediatamente (o registro é apagado do banco), e o identificador fica em um cookie `HttpOnly`, inacessível ao JavaScript da página.

- **Senhas:** hash **bcrypt** com custo 12 (configurável em `BCRYPT_COST`) e salt aleatório embutido. A senha em texto nunca é gravada, registrada em log ou devolvida pela API. Política: 8+ caracteres com letras e números, até 72 bytes (limite do bcrypt).
- **Login:** e-mail inexistente e senha errada recebem a mesma resposta (`401 CREDENCIAIS_INVALIDAS`) e o mesmo tempo de processamento, pois o hash é comparado mesmo quando o usuário não existe. Isso impede descobrir quais e-mails estão cadastrados.
- **Fixação de sessão:** o id de sessão é regenerado a cada login.
- **Cookie:** `cuphead.sid` com `HttpOnly`, `SameSite=Strict` e `Secure` quando servido por HTTPS; expira após 120 minutos sem uso.
- **Armazenamento:** sessões na tabela `sessoes` do PostgreSQL; sobrevivem ao reinício do servidor e são removidas no logout.
- **Usuário recarregado a cada requisição:** a sessão guarda só o id; perfil e existência da conta são lidos do banco, então uma mudança de perfil vale na hora.
- **Força bruta:** *rate limit* de 5 tentativas malsucedidas por IP a cada 15 minutos (`429`).
- **Logout:** `POST /api/auth/logout` destrói a sessão e apaga o cookie; o cookie antigo passa a receber `401`.

6.1 Controle de acesso e matriz de permissões

São três perfis: **visitante** (sem login), **jogador** (padrão no cadastro) e **administrador**. A matriz fica em um único arquivo, `src/domain/permissoes.js`, e cada rota declara a ação que exige, por exemplo `acesso.exigirPermissao('chefe:criar')`. Visitante sem permissão recebe `401`; usuário logado sem permissão recebe `403`. Ações não listadas são negadas por padrão.

Tabela 5: Matriz de permissões

Ação (identificador na matriz)	Visitante	Jogador	Admin
Listar e ver chefes e ilhas (`chefe:listar`)	✓	✓	✓
Criar, editar e excluir chefe (`chefe:criar/editar/excluir`)	–	–	✓
Ver táticas aprovadas (`tatica:listar_aprovadas`)	✓	✓	✓
Enviar tática (`tatica:criar`)	–	✓	✓
Ver as próprias táticas (`tatica:listar_proprias`)	–	✓	✓
Editar/excluir a própria tática pendente (`tatica:editar_propria`)	–	✓	✓
Moderar táticas (`tatica:moderar`)	–	–	✓
Ver conquistas (`conquista:listar`)	✓	✓	✓
Marcar o próprio progresso (`conquista:marcar_progresso`)	–	✓	✓
Listar usuários (`usuario:listar`)	–	–	✓
Alterar perfil de outro usuário (`usuario:alterar_perfil`)	–	–	✓

Regras que dependem do registro, e não só do perfil, ficam no service: o jogador só altera táticas das quais é autor e apenas enquanto estão pendentes (`403`/`409`); o administrador não pode alterar o próprio perfil, para não perder o acesso por engano.

7 Estratégia de segurança

Tabela 6: Ameaças e mitigações

Ameaça	Mitigação	Onde
SQL Injection	Todas as consultas usam parâmetros posicionais (`$1, $2`); o texto SQL nunca é concatenado com entrada do usuário. Curingas do `LIKE` são escapados. Ids de rota são validados como inteiros.	`repositories/`, `validacao.js`
XSS refletido e armazenado	A API responde só JSON com `X-Content-Type-Options: nosniff`. O frontend insere dados apenas com `textContent`/nós de texto (nenhum `innerHTML`). *Content Security Policy* com `script-src 'self'` bloqueia scripts e estilos inline.	`js/dom.js`, `app.js`

Ameaça	Mitigação	Onde
CSRF	Token de 256 bits por sessão, exigido no cabeçalho `X-CSRF-Token` em POST/PUT/PATCH/DELETE e comparado em tempo constante; cookie `SameSite=Strict`.	`middlewares/csrf.js`
Acesso não autorizado	Middleware de permissão em todas as rotas protegidas; regras de autoria no service.	`autorizacao.js`
Mass assignment	A validação só aceita campos descritos no esquema; enviar `"perfil":"admin"` no cadastro é ignorado.	`validacao.js`
Força bruta	*Rate limit* no login e na API.	`limitadores.js`
Vazamento de informação	Stack trace só em desenvolvimento; respostas nunca incluem `senha_hash`; mensagens de login genéricas; cabeçalho `X-Powered-By` removido.	`tratadorErros.js`
Entradas abusivas	Corpo limitado a 10 KB; limites de tamanho em todos os campos; remoção de caracteres de controle e invisíveis.	`app.js`, `validacao.js`
Credenciais no código	Banco, segredo de sessão e administrador vêm do `.env`, que está no `.gitignore`; o repositório tem só o `.env.example`.	`config/env.js`
Clickjacking	`frame-ancestors 'none'` e `X-Frame-Options`.	`app.js`

7.1 Validação e sanitização

Toda entrada externa (corpo, query string e parâmetros de rota) passa pela função `validar(dados, esquema)`, chamada pelos services. Cada campo declara tipo (`texto`, `inteiro`, `email`, `senha`, `opcao`), se é obrigatório e os limites de tamanho ou valor. Textos são normalizados (Unicode NFC), têm caracteres de controle e invisíveis removidos e espaços repetidos reduzidos; senhas não são alteradas. Os erros são devolvidos juntos, por campo:

```
{ "erro": { "status": 422, "codigo": "VALIDACAO",
    "mensagem": "Os dados enviados são inválidos.",
    "detalhes": [ { "campo": "nome",  "mensagem": "deve ser um texto." },
                  { "campo": "email", "mensagem": "deve ser um e-mail válido." },
                  { "campo": "senha", "mensagem": "deve ter no mínimo 8 caracteres." } ] }
     }
```

O banco repete as regras principais em constraints `CHECK`, como última linha de defesa; violações que chegarem até ele também são convertidas em respostas `409` ou `422` pelo tratador de erros.

7.2 Tratamento de erros

Todas as falhas terminam no middleware `tratadorErros.js`, que devolve status HTTP, código, mensagem legível e o `requisicaoId` (o mesmo registrado no log). Erros esperados usam a classe `AppError`; erros do banco e do parser JSON são convertidos; os demais viram `500 ERRO_INTERNO`. O bloco `tecnico` (nome, mensagem original e stack trace) só é incluído com `NODE_ENV=development` (Figura 4); um teste automatizado garante que ele não aparece fora desse modo.

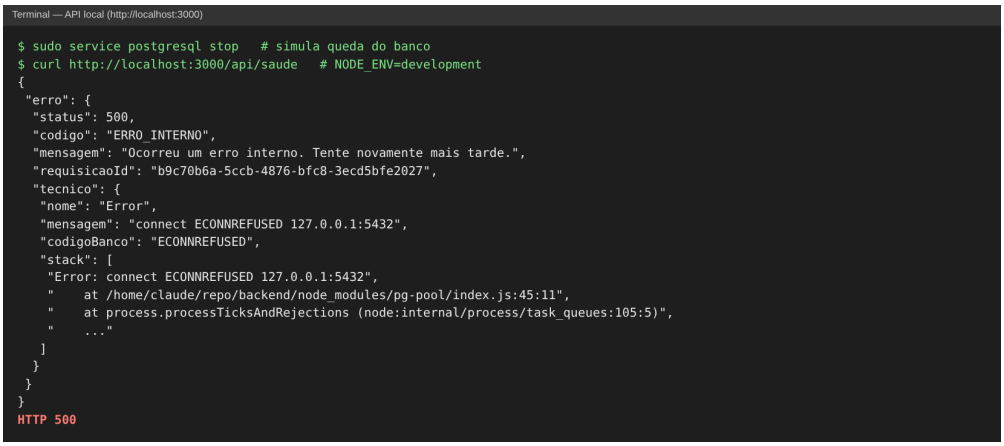


Figura 4: Erro interno em modo desenvolvimento (banco desligado de propósito): detalhes técnicos visíveis

8 Endpoints implementados

A API possui 25 endpoints. A documentação completa (parâmetros, exemplos de requisição e resposta e códigos de status de cada um) está em `docs/API.md` e reproduzida no Anexo B.

Tabela 7: Resumo dos endpoints

Método	URL	Acesso	Sucesso / erros
GET	`/api/saude`	público	200 / 500
GET	`/api/auth/sessao`	público	200
POST	`/api/auth/registro`	público	201 / 403, 409, 422
POST	`/api/auth/login`	público	200 / 401, 403, 422, 429
POST	`/api/auth/logout`	logado	204 / 401, 403
GET	`/api/auth/me`	logado	200 / 401
GET	`/api/ilhas`	público	200
GET	`/api/chefes`	público	200 / 422
GET	`/api/chefes/:id`	público	200 / 404, 422

Método	URL	Acesso	Sucesso / erros
POST	`/api/chefes`	admin	201 / 401, 403, 409, 422
PUT	`/api/chefes/:id`	admin	200 / 401, 403, 404, 409, 422
DELETE	`/api/chefes/:id`	admin	204 / 401, 403, 404
GET	`/api/chefes/:id/taticas`	público	200 / 404, 422
POST	`/api/taticas`	jogador, admin	201 / 401, 403, 422
GET	`/api/taticas/minhas`	jogador, admin	200 / 401
PATCH	`/api/taticas/:id`	autor	200 / 401, 403, 404, 409, 422
DELETE	`/api/taticas/:id`	autor ou admin	204 / 401, 403, 404, 409
GET	`/api/taticas`	admin	200 / 401, 403, 422
PATCH	`/api/taticas/:id/status`	admin	200 / 401, 403, 404, 422
GET	`/api/conquistas`	público	200
GET	`/api/conquistas/progresso`	jogador, admin	200 / 401
PUT	`/api/conquistas/:id/progresso`	jogador, admin	204 / 401, 404, 422
DELETE	`/api/conquistas/:id/progresso`	jogador, admin	204 / 401, 404, 422
GET	`/api/usuarios`	admin	200 / 401, 403, 422
PATCH	`/api/usuarios/:id/perfil`	admin	200 / 401, 403, 404, 409, 422

Os métodos seguem a semântica HTTP: `GET` para leitura, `POST` para criar (com cabeçalho `Location`), `PUT` para substituir, `PATCH` para alterações parciais e `DELETE` para remover (`204` sem corpo). Toda resposta com corpo é JSON no formato `{ "dados": ... }` ou `{ "erro": ... }`. Listagens aceitam `?pagina=` e `?limite=` e devolvem o bloco `paginacao`.

9 Estratégia de testes

Os testes usam o executor nativo do Node.js (`node:test`) e a biblioteca **supertest**, que faz requisições HTTP reais à aplicação Express. São dois níveis:

- **Unitários (34 testes, `tests/unit/`):** testam a lógica de negócio sem banco de dados. Os services recebem repositórios falsos em memória, um *hasher* instantâneo e um logger que captura eventos. Cobrem validação e sanitização, matriz de permissões, `AuthService`, `ChefeService` e `TaticaService`.
- **Integração (22 testes, `tests/integracao/`):** sobem a aplicação completa contra um banco de testes separado (`cuphead_guide_test`), recriado com as mesmas migrações a cada execução. Cobrem CRUD de chefes, fluxo de moderação de táticas, progresso de conquistas, formato dos erros e os testes de segurança.

Os testes cobrem casos válidos e inválidos: dados fora do limite, tipos errados, e-mail duplicado, nome de chefe duplicado, ilha inexistente, senha incorreta, edição de tática já moderada, tática de outro usuário, JSON malformado e rota inexistente, entre outros. A execução completa leva cerca de 3 segundos:

```
$ cd backend
$ npm test               # todos (56)
$ npm run test:unit      # só unitários, sem banco
$ npm run test:relatorio # saída detalhada usada no Anexo C
```

Tabela 8: Resultado da execução dos testes automatizados

Suíte	Testes	Aprovados
Validação e sanitização (unitário)	9	9
Matriz de permissões (unitário)	5	5
AuthService (unitário)	6	6
ChefeService (unitário)	7	7
TaticaService (unitário)	7	7
API – fluxos principais (integração)	8	8
Segurança (integração)	11	11
Rate limiting do login (integração)	1	1
Tratamento de erros (integração)	2	2
Total	**56**	**56**

A saída completa da execução está no Anexo C.

10 Resultados dos testes de segurança

Os testes de segurança foram feitos de duas formas: automatizados (suíte "Segurança", executada a cada `npm test`) e manuais, com `curl` contra o servidor local, pelo script `backend/scripts/demonstrar-seguranca.sh`, que gera as capturas abaixo. A Tabela 9 resume os resultados.

Tabela 9: Testes de segurança

Teste	Ataque	Esperado	Obtido
1. SQL Injection (login)	`' OR '1'='1' --` no e-mail e na senha	Não autenticar	`422` (e-mail inválido) e `401` (chegou ao banco como texto)
1b. SQL Injection (busca)	`'; DROP TABLE chefes; --`	Tratar como texto	`200`, lista vazia; tabela intacta (19 chefes)
2. XSS	Tag `<script>` em uma tática e `<img onerror>` na busca	Não executar	Exibido como texto; nenhum `alert`; JSON com `nosniff` e CSP
3. CSRF	POST com cookie válido e sem token	Bloquear	`403 CSRF_INVALIDO`
4. Acesso sem login	`GET /api/taticas/minhas`	Bloquear	`401 NAO_AUTENTICADO`
5. Perfil insuficiente	Jogador: `DELETE /api/chefes/1`, `GET /api/usuarios`	Bloquear	`403 ACESSO_NEGADO`
6. Sessão inválida	Reusar cookie após logout	Bloquear	`401`
7. Força bruta	Várias senhas erradas	Limitar	`429` após 5 falhas



Figura 5: Teste 1: SQL Injection no login. A primeira tentativa é barrada pela validação; a segunda passa na validação, chega ao banco como parâmetro e não autentica



Figura 6: Teste 1b: SQL Injection na busca. O texto é procurado literalmente e a tabela continua com 19 chefes

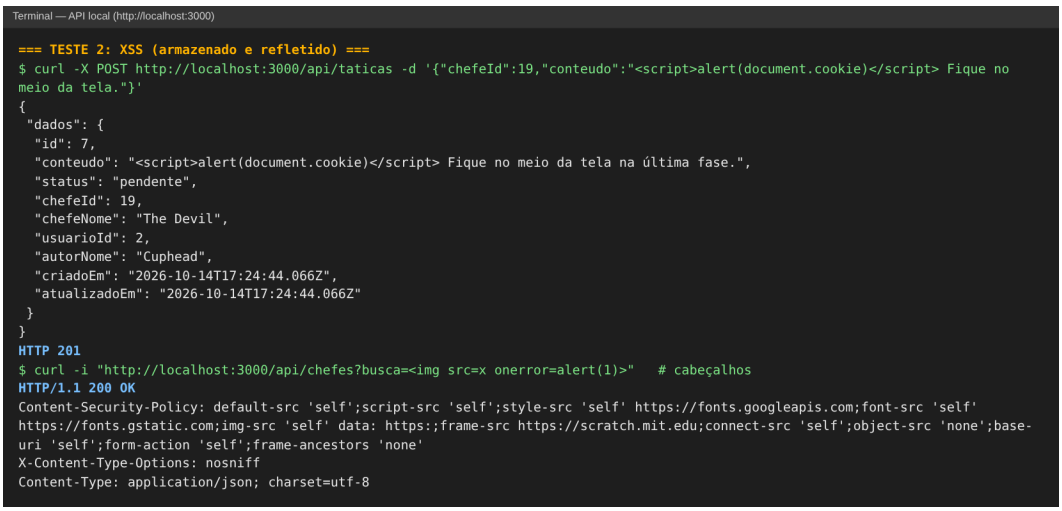


Figura 7: Teste 2: XSS. O conteúdo é aceito e armazenado como texto; respostas são JSON com `nosniff` e CSP

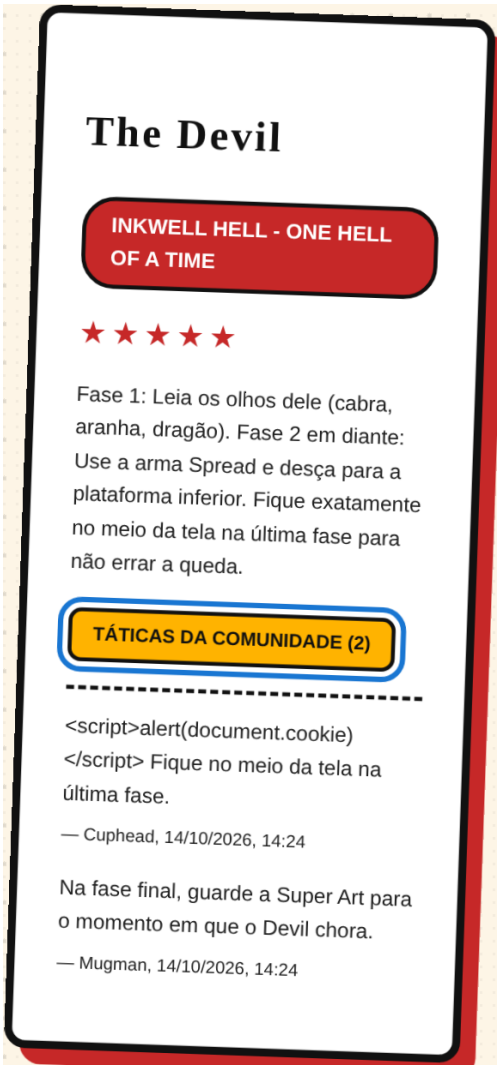


Figura 8: Teste 2 no navegador: a tática maliciosa aparece como texto e nenhum `alert` é disparado



Figura 9: Teste 5 na interface: jogador abrindo `admin.html`. A API também responde `403`

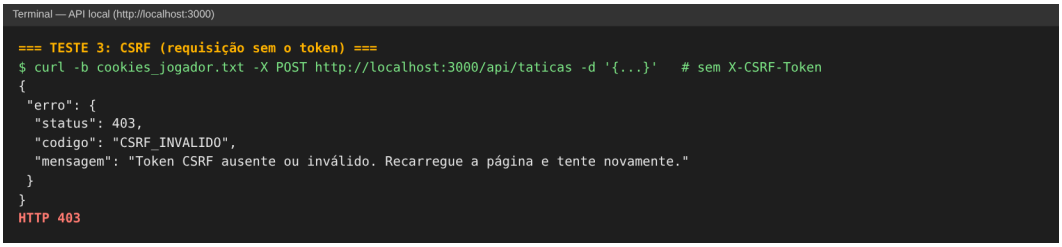


Figura 10: Teste 3: CSRF. Requisição com o cookie de sessão válido, mas sem o token, é recusada

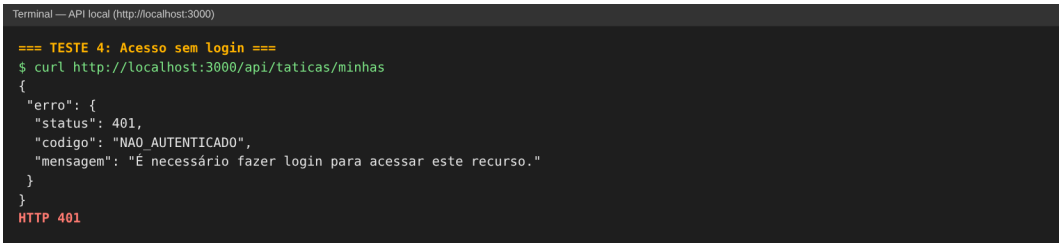


Figura 11: Teste 4: acesso a rota protegida sem login



Figura 12: Teste 5: jogador tentando excluir chefe e listar usuários



Figura 13: Teste 6: o cookie guardado antes do logout não dá mais acesso

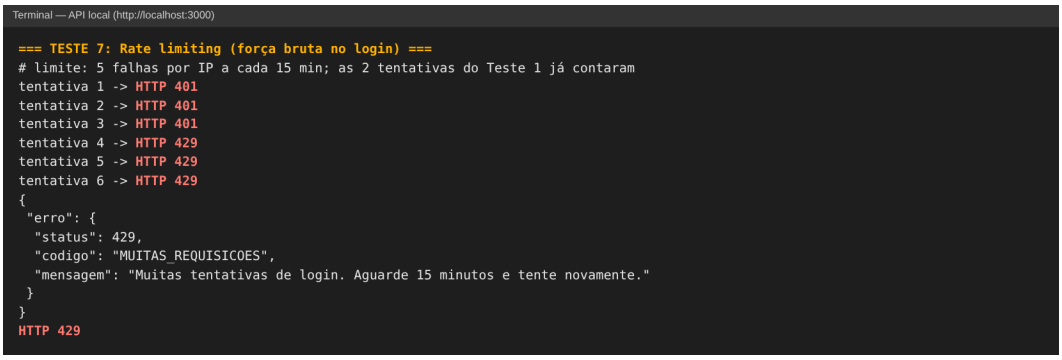


Figura 14: Teste 7: rate limiting. O limite é de 5 falhas por IP em 15 minutos; as 2 tentativas do Teste 1 já contaram

Além desses, a suíte automatizada verifica que o cadastro ignora `"perfil": "admin"` (*mass assignment*), que o banco guarda apenas hashes bcrypt (`$2b$...`), que nenhuma resposta contém o campo de senha e que o stack trace não aparece fora do modo de desenvolvimento.

11 Logs e observabilidade

O módulo `config/logger.js` grava cada evento como uma linha JSON no arquivo `backend/logs/app.log` (e no console). Toda linha tem `timestamp` no formato ISO-8601 (UTC), nível (`debug`, `info`, `aviso`, `erro`), nome do evento e contexto. Campos como `senha`, `token` e `cookie` são descartados automaticamente antes da gravação.

Tabela 10: Eventos registrados

Evento	Nível	Quando
`auth.login_sucesso`	info	Login aceito (usuário, perfil, IP)
`auth.login_falha`	aviso	Credenciais erradas (e-mail, IP, motivo)
`auth.logout`	info	Sessão encerrada
`entidade.criada`	info	Usuário, chefe ou tática criados
`entidade.atualizada` / `excluida`	info	Alterações e exclusões
`erro.interno`	erro	Falha inesperada (mensagem, stack, rota, `requisicaoId`)
`acesso.negado`	aviso	Respostas 401/403 por falta de permissão
`seguranca.csrf_invalido`	aviso	Token CSRF ausente ou diferente
`seguranca.rate_limit`	aviso	Limite de requisições atingido
`servidor.iniciado` / `encerrando`	info	Ciclo de vida do processo
`http.requisicao`	debug	Método, rota, status e duração de cada requisição

Cada requisição recebe um identificador único, devolvido no cabeçalho `X-Request-Id` e no corpo de qualquer erro. Assim, quando um usuário relata um erro 500, o `requisicaoId` da resposta leva direto à linha do log com a causa. Linhas reais do log (trecho do Anexo E; a terceira é a tentativa de SQL Injection do Teste 1, registrada como e-mail inexistente):


```
{"timestamp":"2026-10-14T17:24:15.001Z","nivel":"info","evento":"auth.login_sucesso","
    usuarioId":2,"perfil":"jogador","ip":"127.0.0.1"}
{"timestamp":"2026-10-14T17:24:17.781Z","nivel":"aviso","evento":"auth.login_falha","email
    ":"admin@cuphead.local","ip":"127.0.0.1","motivo":"senha_incorreta"}
{"timestamp":"2026-10-14T17:24:32.592Z","nivel":"aviso","evento":"auth.login_falha","email
    ":"admin@cuphead.local'--","ip":"127.0.0.1","motivo":"usuario_inexistente"}
{"timestamp":"2026-10-14T17:24:15.045Z","nivel":"info","evento":"entidade.criada","
    entidade":"tatica","id":1,"chefeId":1,"por":2}
{"timestamp":"2026-10-14T17:27:56.882Z","nivel":"erro","evento":"erro.interno","
    requisicaoId":"b9c70b6a-5ccb-4876-bfc8-3ecd5bfe2027","metodo":"GET","rota":"/api/saude
    ","usuarioId":null,"mensagem":"connect ECONNREFUSED 127.0.0.1:5432"}
```

12 Desempenho e limitações

12.1 Medição de desempenho

O script `npm run bench` sobe a aplicação com o banco de desenvolvimento e usa o **autocannon** para disparar carga contra os endpoints de leitura mais usados (10 conexões simultâneas, 5 segundos por endpoint). Também mede o tempo do bcrypt. Ambiente: Node.js 22, PostgreSQL 16, mesma máquina.

Tabela 11: Resultados do benchmark

Endpoint	Req/s	Média (ms)	p50 (ms)	p99 (ms)	Erros
`GET /api/chefes`	537	18,1	16	45	0
`GET /api/chefes?ilha=2&busca=the`	761	12,6	11	35	0
`GET /api/chefes/1`	996	9,5	9	22	0
`GET /api/chefes/1/taticas`	975	9,8	9	19	0
`GET /api/conquistas`	1219	7,7	7	16	0

Todas as leituras ficaram abaixo de 50 ms no percentil 99, sem erros. A listagem de chefes é a mais lenta porque executa duas consultas (contagem total para a paginação e a página em si) e uma subconsulta que conta as táticas aprovadas de cada chefe.

O login é intencionalmente lento: com custo 12, o bcrypt levou cerca de 460 ms para verificar uma senha. Esse custo é o que torna inviável testar milhões de senhas caso o banco vaze, e foi mantido.

12.2 Limitações

- O *rate limit* é guardado em memória: reiniciar o servidor zera os contadores, e com mais de uma instância cada uma teria o seu. Uma versão em produção usaria um armazenamento compartilhado (por exemplo, Redis).
- A biblioteca `bcryptjs` é JavaScript puro, escolhida para instalar no Windows sem compilador; a versão nativa (`bcrypt`) seria cerca de três vezes mais rápida.
- Localmente o sistema roda em HTTP. Em produção seria necessário HTTPS (e `COOKIE_SECURE=true`).
- Não há confirmação de e-mail nem recuperação de senha.
- As imagens da galeria continuam vindo de sites externos, como no Trabalho I; se algum deles sair do ar, a imagem some.
- A contagem de táticas por chefe é feita em uma subconsulta a cada listagem; com muitos dados, valeria guardar esse número em uma coluna.

- As métricas do Google Lighthouse medidas no Trabalho I não foram refeitas para este relatório.

13 Uso de IA generativa

Ferramenta utilizada: Claude (Anthropic), em uma sessão de agente com acesso ao repositório, ao PostgreSQL e ao terminal. No Trabalho I havia sido usado o Google Gemini.

Finalidade e partes produzidas: a ferramenta recebeu o enunciado do Trabalho II, o relatório e o código do Trabalho I e foi usada para planejar e gerar a maior parte do Trabalho II:

- backend completo (configuração, middlewares, rotas, controllers, services, repositories e validação);
- modelo de dados, migrações e script SQL;
- JavaScript do frontend, páginas `conta.html` e `admin.html` e os estilos novos, seguindo a identidade visual do Trabalho I;
- testes automatizados, script de benchmark e script de demonstração de segurança;
- documentação (`README.md`, `docs/API.md`, `docs/ARQUITETURA.md`, `docs/CHECKLIST.md`), capturas de tela e o texto deste relatório.

Rastreabilidade: todos os commits do Trabalho II, a partir de `chore: reorganiza o projeto do T1 na pasta frontend/` (Anexo F), foram produzidos com apoio da ferramenta. Os commits do Trabalho I (até a tag `v1.0`) não foram alterados.

Problemas encontrados e corrigidos durante o desenvolvimento: parte do que a IA gerou precisou de correção depois de executado e testado:

- a senha do administrador no `.env` continha `#`, que o `dotenv` interpreta como início de comentário; a senha era cortada e o script `criar-admin` falhava. Foi adicionada uma observação no `.env.example`;
- o *reset* das migrações usava `DROP SCHEMA public`, que falha em versões do PostgreSQL anteriores à 15 quando o usuário não é dono do schema. Foi trocado pela remoção explícita das tabelas do projeto (commit `fix(db)`);
- a primeira versão da migração de dados inseria os chefes fora da ordem do guia original;
- mensagens como "Conquista não encontrado" tinham erro de concordância;
- os logs registravam a rota relativa ao roteador (`/`) em vez da rota completa (`/api/usuarios`);
- no celular, o cabeçalho fixo passou a ocupar quase metade da tela depois da inclusão da área de login (commit `fix(frontend)`).

Avaliação crítica: a IA acelerou muito a produção de código repetitivo (camadas, documentação de endpoints, testes) e sugeriu proteções que não estavam explícitas no enunciado, como comparação de senha em tempo constante e *Content Security Policy*. Por outro lado, vários dos problemas acima só apareceram quando o sistema foi executado de verdade, o que mostra que o código gerado não pode ser aceito sem execução e testes. A responsabilidade pela revisão, pela execução local e pela apresentação do sistema é do estudante.

14 Conclusões

O *Cuphead Boss Guide* deixou de ser uma página estática e passou a ser uma aplicação cliente-servidor com dados persistentes, usuários com perfis distintos e fluxo de colaboração (envio e moderação de táticas). As duas limitações apontadas no Trabalho I, o formulário sem efeito e o checklist que não salvava, foram resolvidas.

Do ponto de vista de engenharia, a separação em camadas com injeção de dependências foi o que permitiu testar a lógica de negócio isoladamente e, ao mesmo tempo, testar a aplicação completa contra um banco real. As proteções exigidas (consultas parametrizadas, escape de saída, CSRF e controle de acesso) foram implementadas e comprovadas por testes automatizados e manuais, e o tratamento centralizado de erros com identificador de requisição liga cada resposta de erro à sua linha no log.

Como próximos passos, ficam: HTTPS e implantação em um servidor, *rate limit* com armazenamento compartilhado, recuperação de senha por e-mail e uma nova auditoria Lighthouse da interface com os componentes dinâmicos.

Anexos

A Script de criação do banco

Arquivo completo: `backend/database/script_completo.sql`. Abaixo, a migração de estrutura (`001_criar_tabelas.sql`); a migração 002 contém os `INSERT` dos 19 chefes e 28 conquistas.

```sql
-- ============================================================================
-- Migração 001 - Criação das tabelas do Cuphead Boss Guide
-- SGBD: PostgreSQL 12+ (testado na versão 16)
-- ============================================================================

-- ----------------------------------------------------------------------------
-- Usuários e perfis de acesso
-- ----------------------------------------------------------------------------
CREATE TABLE usuarios (
    id          INTEGER GENERATED ALWAYS AS IDENTITY,
    nome        VARCHAR(60)  NOT NULL,
    email       VARCHAR(254) NOT NULL,
    senha_hash  VARCHAR(72)  NOT NULL,          -- hash bcrypt (60 caracteres)
    perfil      VARCHAR(10)  NOT NULL DEFAULT 'jogador',
    criado_em   TIMESTAMPTZ  NOT NULL DEFAULT now(),

    CONSTRAINT pk_usuarios PRIMARY KEY (id),
    CONSTRAINT uq_usuarios_email UNIQUE (email),
    CONSTRAINT ck_usuarios_perfil CHECK (perfil IN ('jogador', 'admin')),
    CONSTRAINT ck_usuarios_email_minusculo CHECK (email = lower(email)),
    CONSTRAINT ck_usuarios_nome_tamanho CHECK (char_length(nome) BETWEEN 2 AND 60)
);

-- ----------------------------------------------------------------------------
-- Ilhas (mundos) do jogo
-- ----------------------------------------------------------------------------
CREATE TABLE ilhas (
    id     SMALLINT GENERATED ALWAYS AS IDENTITY,
    nome   VARCHAR(40) NOT NULL,
    ordem  SMALLINT    NOT NULL,

    CONSTRAINT pk_ilhas PRIMARY KEY (id),
    CONSTRAINT uq_ilhas_nome UNIQUE (nome),
    CONSTRAINT uq_ilhas_ordem UNIQUE (ordem),
    CONSTRAINT ck_ilhas_ordem_positiva CHECK (ordem > 0)
);

-- ----------------------------------------------------------------------------
-- Chefes (entidade principal)
-- ----------------------------------------------------------------------------
CREATE TABLE chefes (
    id             INTEGER GENERATED ALWAYS AS IDENTITY,
    nome           VARCHAR(60)  NOT NULL,
    fase           VARCHAR(80)  NOT NULL,          -- nome da fase/nível no jogo
    ilha_id        SMALLINT     NOT NULL,
    estrategia     VARCHAR(1000) NOT NULL,
    dificuldade    SMALLINT     NOT NULL,
    criado_por     INTEGER,
    criado_em      TIMESTAMPTZ  NOT NULL DEFAULT now(),
    atualizado_em  TIMESTAMPTZ  NOT NULL DEFAULT now(),

    CONSTRAINT pk_chefes PRIMARY KEY (id),
    CONSTRAINT uq_chefes_nome UNIQUE (nome),
    CONSTRAINT fk_chefes_ilha FOREIGN KEY (ilha_id)
        REFERENCES ilhas (id) ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_chefes_criado_por FOREIGN KEY (criado_por)
        REFERENCES usuarios (id) ON DELETE SET NULL,
    CONSTRAINT ck_chefes_dificuldade CHECK (dificuldade BETWEEN 1 AND 5),
    CONSTRAINT ck_chefes_estrategia_tamanho CHECK (char_length(estrategia) BETWEEN 20 AND 1000)
```

```
);

-- Listagem filtrada por ilha (consulta mais frequente do site)
CREATE INDEX idx_chefes_ilha_id ON chefes (ilha_id);
-- Busca por nome sem diferenciar maiúsculas/minúsculas
CREATE INDEX idx_chefes_nome_lower ON chefes (lower(nome));

-- ---------------------------------------------------------------------
-- Táticas sugeridas pela comunidade (moderadas pelo administrador)
-- ---------------------------------------------------------------------
CREATE TABLE taticas (
    id             INTEGER GENERATED ALWAYS AS IDENTITY,
    chefe_id       INTEGER      NOT NULL,
    usuario_id     INTEGER      NOT NULL,
    conteudo       VARCHAR(1000) NOT NULL,
    status         VARCHAR(10)  NOT NULL DEFAULT 'pendente',
    moderado_por   INTEGER,
    criado_em      TIMESTAMPTZ  NOT NULL DEFAULT now(),
    atualizado_em  TIMESTAMPTZ  NOT NULL DEFAULT now(),

    CONSTRAINT pk_taticas PRIMARY KEY (id),
    CONSTRAINT fk_taticas_chefe FOREIGN KEY (chefe_id)
        REFERENCES chefes (id) ON DELETE CASCADE,
    CONSTRAINT fk_taticas_usuario FOREIGN KEY (usuario_id)
        REFERENCES usuarios (id) ON DELETE CASCADE,
    CONSTRAINT fk_taticas_moderado_por FOREIGN KEY (moderado_por)
        REFERENCES usuarios (id) ON DELETE SET NULL,
    CONSTRAINT ck_taticas_status CHECK (status IN ('pendente', 'aprovada', 'rejeitada')),
    CONSTRAINT ck_taticas_conteudo_tamanho CHECK (char_length(conteudo) BETWEEN 10 AND 1000)
);

-- Táticas aprovadas de um chefe (página pública)
CREATE INDEX idx_taticas_chefe_status ON taticas (chefe_id, status);
-- "Minhas táticas"
CREATE INDEX idx_taticas_usuario ON taticas (usuario_id);
-- Fila de moderação ordenada por data
CREATE INDEX idx_taticas_status_criado_em ON taticas (status, criado_em);

-- ---------------------------------------------------------------------
-- Conquistas do jogo e progresso de cada usuário
-- ---------------------------------------------------------------------
CREATE TABLE conquistas (
    id         SMALLINT GENERATED ALWAYS AS IDENTITY,
    nome       VARCHAR(60)  NOT NULL,
    descricao  VARCHAR(200) NOT NULL,
    ordem      SMALLINT     NOT NULL,

    CONSTRAINT pk_conquistas PRIMARY KEY (id),
    CONSTRAINT uq_conquistas_nome UNIQUE (nome),
    CONSTRAINT uq_conquistas_ordem UNIQUE (ordem)
);

CREATE TABLE usuario_conquistas (
    usuario_id    INTEGER     NOT NULL,
    conquista_id  SMALLINT    NOT NULL,
    concluida_em  TIMESTAMPTZ NOT NULL DEFAULT now(),

    CONSTRAINT pk_usuario_conquistas PRIMARY KEY (usuario_id, conquista_id),
    CONSTRAINT fk_usuario_conquistas_usuario FOREIGN KEY (usuario_id)
        REFERENCES usuarios (id) ON DELETE CASCADE,
    CONSTRAINT fk_usuario_conquistas_conquista FOREIGN KEY (conquista_id)
        REFERENCES conquistas (id) ON DELETE CASCADE
);

-- ---------------------------------------------------------------------
-- Sessões de login (usada pelo connect-pg-simple)
-- ---------------------------------------------------------------------
CREATE TABLE sessoes (
    sid     VARCHAR      NOT NULL,
    sess    JSON         NOT NULL,
    expire  TIMESTAMP(6) NOT NULL,
```

```
    CONSTRAINT pk_sessoes PRIMARY KEY (sid)
);

-- Limpeza periódica de sessões expiradas
CREATE INDEX idx_sessoes_expire ON sessoes (expire);

-- ---------------------------------------------------------------------
-- Atualiza "atualizado_em" automaticamente em todo UPDATE
-- ---------------------------------------------------------------------
CREATE FUNCTION definir_atualizado_em() RETURNS trigger AS $$
BEGIN
    NEW.atualizado_em := now();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_chefes_atualizado_em
    BEFORE UPDATE ON chefes
    FOR EACH ROW EXECUTE FUNCTION definir_atualizado_em();

CREATE TRIGGER trg_taticas_atualizado_em
    BEFORE UPDATE ON taticas
    FOR EACH ROW EXECUTE FUNCTION definir_atualizado_em();
```

B Documentação da API

B.1 Documentação da API — Cuphead Boss Guide

API REST em JSON servida em `http://localhost:3000/api`.

B.1.1 Convenções

Autenticação e sessão

- A autenticação é por **sessão** (cookie `cuphead.sid`, `HttpOnly`, `SameSite=Strict`).
- O navegador envia o cookie automaticamente; ferramentas como `curl` precisam guardá-lo (`-c cookies.txt -b cookies.txt`).

Proteção CSRF

Toda requisição `POST`, `PUT`, `PATCH` ou `DELETE` precisa do cabeçalho `X-CSRF-Token` com o token obtido em `GET /api/auth/sessao`. O login devolve um **token novo**, que deve substituir o anterior.

Formato das respostas

Sucesso:

```
{ "dados": { ... } }
```

Listagens paginadas incluem `paginacao`:

```
{ "dados": [ ... ], "paginacao": { "pagina": 1, "limite": 6, "total": 19, "totalPaginas": 4 } }
```

Erro (todas as rotas):

```
{
  "erro": {
    "status": 422,
    "codigo": "VALIDACAO",
    "mensagem": "Os dados enviados são inválidos.",
    "requisicaoId": "2df0b9ae-bcc7-4214-8fa4-eb84460f3315",
    "detalhes": [{ "campo": "dificuldade", "mensagem": "deve ser menor ou igual a 5." }]
  }
}
```

Com `NODE_ENV=development`, erros inesperados (500) também trazem o bloco `tecnico` (nome, mensagem original e stack trace). Em produção esse bloco nunca aparece.

Códigos de status usados

Status	Código (`erro.codigo`)	Quando
200	—	Leitura ou atualização bem-sucedida
201	—	Recurso criado (cabeçalho `Location` aponta para ele)
204	—	Sucesso sem corpo (logout, exclusão, marcar conquista)
400	`JSON_INVALIDO`	Corpo da requisição não é JSON válido
401	`NAO_AUTENTICADO`	Rota exige login
401	`CREDENCIAIS_INVALIDAS`	E-mail ou senha incorretos
403	`ACESSO_NEGADO`	Perfil sem permissão (ver matriz em `ARQUITETURA.md`)
403	`CSRF_INVALIDO`	Cabeçalho `X-CSRF-Token` ausente ou incorreto
404	`NAO_ENCONTRADO` / `ROTA_NAO_ENCONTRADA`	Registro ou rota inexistente
409	`CONFLITO`	E-mail/nome duplicado, tática já moderada, alterar o próprio perfil
413	`CORPO_MUITO_GRANDE`	Corpo maior que 10 KB
422	`VALIDACAO`	Tipo, tamanho ou formato inválido (lista em `detalhes`)
429	`MUITAS_REQUISICOES`	Rate limit excedido (login ou API)
500	`ERRO_INTERNO`	Falha inesperada (registrada no log com `requisicaoId`)

Perfis

`visitante` (sem login), `jogador` (padrão no cadastro) e `admin`.

B.1.2 Índice

Método	URL	Acesso
GET	/api/saude	público
GET	/api/auth/sessao	público
POST	/api/auth/registro	público
POST	/api/auth/login	público (rate limit)
POST	/api/auth/logout	logado
GET	/api/auth/me	logado
GET	/api/ilhas	público
GET	/api/chefes	público
GET	/api/chefes/:id	público
POST	/api/chefes	admin
PUT	/api/chefes/:id	admin
DELETE	/api/chefes/:id	admin
GET	/api/chefes/:id/taticas	público
POST	/api/taticas	jogador, admin
GET	/api/taticas/minhas	jogador, admin
PATCH	/api/taticas/:id	autor (tática pendente)
DELETE	/api/taticas/:id	autor (pendente) ou admin
GET	/api/taticas	admin
PATCH	/api/taticas/:id/status	admin
GET	/api/conquistas	público
GET	/api/conquistas/progresso	jogador, admin
PUT	/api/conquistas/:id/progresso	jogador, admin
DELETE	/api/conquistas/:id/progresso	jogador, admin
GET	/api/usuarios	admin
PATCH	/api/usuarios/:id/perfil	admin

B.1.3 Saúde

`GET /api/saude`

Verifica se a API e o banco estão respondendo.

Parâmetros: nenhum.

```
curl http://localhost:3000/api/saude
```

```
{ "dados": { "status": "ok", "banco": "ok" } }
```

Status: `200`, `500` (banco indisponível).

B.1.4 Autenticação

`GET /api/auth/sessao`

Retorna o usuário logado (ou `null`) e o token CSRF da sessão. É a primeira chamada do frontend.

Parâmetros: nenhum.

```
curl -c cookies.txt -b cookies.txt http://localhost:3000/api/auth/sessao
```

```
{ "dados": { "usuario": null, "csrfToken": "I8bheqSPZxh4GORdVYSbqZJvIHBMyNJhKqCkg-5M5AM" } }
```

Status: `200`.

`POST /api/auth/registro`

Cria uma conta com perfil `jogador`. Campos extras (ex.: `perfil`) são ignorados.

Corpo (JSON):

Campo	Tipo	Regras
`nome`	texto	obrigatório, 2–60 caracteres
`email`	texto	obrigatório, e-mail válido, até 254 caracteres (convertido para minúsculas)
`senha`	texto	obrigatório, 8+ caracteres, letras e números, até 72 bytes

```
curl -b cookies.txt -c cookies.txt -X POST http://localhost:3000/api/auth/registro \
  -H "Content-Type: application/json" -H "X-CSRF-Token: <token>" \
  -d '{"nome":"Cuphead","email":"cup@head.com","senha":"xicara123"}'
```

```
{
  "dados": { "id": 2, "nome": "Cuphead", "email": "cup@head.com", "perfil": "jogador", "criadoEm":
  ↪  "2026-10-14T17:13:40.620Z" }
}
```

Status: `201`, `403` (CSRF), `409` (e-mail já cadastrado), `422`.

`POST /api/auth/login`

Autentica e cria a sessão. O id de sessão é regenerado (anti-fixação) e um **novo token CSRF** é devolvido. Limitado a `RATE_LIMIT_LOGIN_MAX` tentativas malsucedidas por IP a cada `RATE_LIMIT_LOGIN_JANELA_MIN` minutos.

Corpo (JSON): `email` (texto, obrigatório), `senha` (texto, obrigatório).

```
curl -b cookies.txt -c cookies.txt -X POST http://localhost:3000/api/auth/login \
  -H "Content-Type: application/json" -H "X-CSRF-Token: <token>" \
  -d '{"email":"cup@head.com","senha":"xicara123"}'
```

```
{
  "dados": {
    "usuario": { "id": 2, "nome": "Cuphead", "email": "cup@head.com", "perfil": "jogador", "criadoEm":
    ↪  "2026-10-14T17:13:40.620Z" },
    "csrfToken": "RMBNkIqvIVZ8zJvx7kQvLUQUd6Mnrq8bDTWoCPp2sp4"
  }
}
```

Credenciais erradas (mesma resposta para e-mail inexistente ou senha incorreta):

```
{ "erro": { "status": 401, "codigo": "CREDENCIAIS_INVALIDAS", "mensagem": "E-mail ou senha incorretos.",
↪  "requisicaoId": "f387e924-299a-423b-a5b2-50c3b2ecc659" } }
```

Status: `200`, `401`, `403` (CSRF), `422`, `429`.

`POST /api/auth/logout`

Destroi a sessão no servidor e apaga o cookie. O cookie antigo deixa de valer.

Parâmetros: nenhum (exige `X-CSRF-Token`).

```
curl -b cookies.txt -c cookies.txt -X POST http://localhost:3000/api/auth/logout -H "X-CSRF-Token: <token>"
```

Resposta: corpo vazio.

Status: `204`, `401`, `403` (CSRF).

`GET /api/auth/me`

Dados do usuário logado.

```
curl -b cookies.txt http://localhost:3000/api/auth/me
```

```
{ "dados": { "id": 2, "nome": "Cuphead", "email": "cup@head.com", "perfil": "jogador", "criadoEm":
↪  "2026-10-14T17:13:40.620Z" } }
```

Status: `200`, `401`.

B.1.5 Ilhas

`GET /api/ilhas`

```
curl http://localhost:3000/api/ilhas
```

```
{
  "dados": [
    { "id": 1, "nome": "Ilha 1", "ordem": 1 },
    { "id": 2, "nome": "Ilha 2", "ordem": 2 },
    { "id": 3, "nome": "Ilha 3", "ordem": 3 },
    { "id": 4, "nome": "Inkwell Hell", "ordem": 4 }
  ]
}
```

Status: `200`.

B.1.6 Chefes

Objeto `Chefe`:

```
{
  "id": 1,
  "nome": "The Root Pack",
  "fase": "Botanic Panic",
  "estrategia": "O FSM transita de forma linear: Batata (projéteis horizontais), ...",
  "dificuldade": 1,
  "ilhaId": 1,
  "ilhaNome": "Ilha 1",
  "criadoEm": "2026-10-14T17:12:36.512Z",
  "atualizadoEm": "2026-10-14T17:12:36.512Z",
  "totalTaticas": 1
}
```

`GET /api/chefes`

Lista paginada, ordenada por ilha.

Query string:

Parâmetro	Tipo	Padrão	Regras
`pagina`	inteiro	1	≥ 1
`limite`	inteiro	6	1–50
`ilha`	inteiro	—	id da ilha
`busca`	texto	—	1–60 caracteres; procura em nome e fase, sem diferenciar maiúsculas

```
curl "http://localhost:3000/api/chefes?ilha=4&busca=dev"
```

```
{
  "dados": [
    { "id": 19, "nome": "The Devil", "fase": "One Hell of a Time", "estrategia": "Fase 1: Leia os olhos dele ...",
    ↪  "dificuldade": 5,
      "ilhaId": 4, "ilhaNome": "Inkwell Hell", "criadoEm": "...", "atualizadoEm": "...", "totalTaticas": 0 }
  ],
  "paginacao": { "pagina": 1, "limite": 6, "total": 1, "totalPaginas": 1 }
}
```

Status: `200`, `422` (ex.: `limite=999`).

`GET /api/chefes/:id`

Parâmetros de rota: `id` (inteiro ≥ 1).

```
curl http://localhost:3000/api/chefes/1
```

Resposta: `{ "dados": Chefe }`.

Status: `200`, `404`, `422` (id não numérico).

`POST /api/chefes`

Somente **admin**.

Corpo (JSON):

Campo	Tipo	Regras
`nome`	texto	obrigatório, 2–60, único (sem diferenciar maiúsculas)
`fase`	texto	obrigatório, 2–80
`ilhaId`	inteiro	obrigatório, ilha existente
`estrategia`	texto	obrigatório, 20–1000, aceita quebras de linha
`dificuldade`	inteiro	obrigatório, 1–5

```
curl -b cookies.txt -X POST http://localhost:3000/api/chefes \
  -H "Content-Type: application/json" -H "X-CSRF-Token: <token>" \
  -d '{"nome":"Glumstone the Giant","fase":"Doggone Dogfight","ilhaId":4,"estrategia":"Atire nos olhos e evite as
  ↪  mãos.","dificuldade":4}'
```

```
{
  "dados": { "id": 20, "nome": "Glumstone the Giant", "fase": "Doggone Dogfight", "estrategia": "Atire nos olhos e
  ↪  evite as mãos.",
            "dificuldade": 4, "ilhaId": 4, "ilhaNome": "Inkwell Hell", "criadoEm": "...", "atualizadoEm": "...",
            ↪  "totalTaticas": 0 }
}
```

Cabeçalho `Location: /api/chefes/20`.

Status: `201`, `401`, `403`, `409` (nome duplicado), `422`.

`PUT /api/chefes/:id`

Somente **admin**. Substitui todos os campos (mesmo corpo do `POST`).

```
curl -b cookies.txt -X PUT http://localhost:3000/api/chefes/20 \
  -H "Content-Type: application/json" -H "X-CSRF-Token: <token>" \
  -d '{"nome":"Glumstone","fase":"Doggone Dogfight","ilhaId":4,"estrategia":"Estratégia atualizada pelo
  ↪  administrador.","dificuldade":3}'
```

Resposta: `{ "dados": Chefe }` atualizado.

Status: `200`, `401`, `403`, `404`, `409`, `422`.

`DELETE /api/chefes/:id`

Somente **admin**. Remove o chefe e, em cascata, as táticas dele.

```
curl -b cookies.txt -X DELETE http://localhost:3000/api/chefes/20 -H "X-CSRF-Token: <token>"
```

Resposta: corpo vazio.

Status: `204`, `401`, `403`, `404`.

`GET /api/chefes/:id/taticas`

Táticas **aprovadas** do chefe, mais recentes primeiro.

```
curl http://localhost:3000/api/chefes/1/taticas
```

```
{
  "dados": [
    { "id": 1, "conteudo": "Fique no canto durante a fase da cenoura.", "status": "aprovada", "chefeId": 1,
    ↪  "chefeNome": "The Root Pack",
      "usuarioId": 2, "autorNome": "Cuphead", "criadoEm": "...", "atualizadoEm": "..." }
  ]
}
```

Status: `200`, `404`, `422`.

B.1.7 Táticas da comunidade

Objeto `Tatica`: `id`, `conteudo`, `status` (`pendente` | `aprovada` | `rejeitada`), `chefeId`, `chefeNome`, `usuarioId`, `autorNome`, `criadoEm`, `atualizadoEm`.

`POST /api/taticas`

Jogador ou admin envia uma tática. Ela nasce `pendente`.

Corpo (JSON): `chefeId` (inteiro, chefe existente), `conteudo` (texto, 10–1000, aceita quebras de linha).

```
curl -b cookies.txt -X POST http://localhost:3000/api/taticas \
  -H "Content-Type: application/json" -H "X-CSRF-Token: <token>" \
  -d '{"chefeId":1,"conteudo":"Fique no canto durante a fase da cenoura."}'
```

```
{
  "dados": { "id": 1, "conteudo": "Fique no canto durante a fase da cenoura.", "status": "pendente", "chefeId": 1,
             "chefeNome": "The Root Pack", "usuarioId": 2, "autorNome": "Cuphead", "criadoEm": "...",
             ↪  "atualizadoEm": "..." }
}
```

Status: `201`, `401`, `403` (CSRF), `422` (inclui chefe inexistente).

`GET /api/taticas/minhas`

Todas as táticas do usuário logado (qualquer status).

```
curl -b cookies.txt http://localhost:3000/api/taticas/minhas
```

Resposta: `{ "dados": [Tatica, ...] }`.

Status: `200`, `401`.

`PATCH /api/taticas/:id`

O **autor** altera o conteúdo enquanto a tática está `pendente`.

Corpo (JSON): `conteudo` (texto, 10–1000).

```
curl -b cookies.txt -X PATCH http://localhost:3000/api/taticas/1 \
  -H "Content-Type: application/json" -H "X-CSRF-Token: <token>" \
  -d '{"conteudo":"Conteúdo editado pelo autor enquanto pendente."}'
```

Resposta: `{ "dados": Tatica }`.

Status: `200`, `401`, `403` (não é o autor), `404`, `409` (já moderada), `422`.

`DELETE /api/taticas/:id`

O autor exclui a própria tática pendente; o **admin** exclui qualquer uma.

```
curl -b cookies.txt -X DELETE http://localhost:3000/api/taticas/1 -H "X-CSRF-Token: <token>"
```

Status: `204`, `401`, `403`, `404`, `409`.

`GET /api/taticas`

Fila de moderação (somente **admin**), mais antigas primeiro.

Query string: `status` (`pendente` | `aprovada` | `rejeitada`, opcional), `pagina`, `limite` (padrão 20, máx. 50).

```
curl -b cookies.txt "http://localhost:3000/api/taticas?status=pendente"
```

Resposta: `{ "dados": [Tatica, ...], "paginacao": { ... } }`.

Status: `200`, `401`, `403`, `422`.

`PATCH /api/taticas/:id/status`

Aprova ou rejeita uma tática (somente **admin**).

Corpo (JSON): `status` (`aprovada` | `rejeitada`).

```
curl -b cookies.txt -X PATCH http://localhost:3000/api/taticas/1/status \
  -H "Content-Type: application/json" -H "X-CSRF-Token: <token>" -d '{"status":"aprovada"}'
```

Resposta: `{ "dados": Tatica }` com o novo status.

Status: `200`, `401`, `403`, `404`, `422`.

B.1.8 Conquistas

`GET /api/conquistas`

Catálogo com as 28 conquistas.

```
curl http://localhost:3000/api/conquistas
```

```
{ "dados": [ { "id": 1, "nome": "Taking Names", "descricao": "Derrote um chefão." }, "..." ] }
```

Status: `200`.

`GET /api/conquistas/progresso`

Progresso do usuário logado.

```
curl -b cookies.txt http://localhost:3000/api/conquistas/progresso
```

```
{ "dados": { "total": 28, "concluidas": 1, "percentual": 4, "itens": [ { "conquistaId": 3, "concluidaEm":
↪  "2026-10-14T17:13:41.244Z" } ] } }
```

Status: `200`, `401`.

`PUT /api/conquistas/:id/progresso`

Marca a conquista como concluída (idempotente: repetir não gera erro).

```
curl -b cookies.txt -X PUT http://localhost:3000/api/conquistas/3/progresso -H "X-CSRF-Token: <token>"
```

Status: `204`, `401`, `403` (CSRF), `404`, `422`.

`DELETE /api/conquistas/:id/progresso`

Desmarca a conquista.

```
curl -b cookies.txt -X DELETE http://localhost:3000/api/conquistas/3/progresso -H "X-CSRF-Token: <token>"
```

Status: `204`, `401`, `403` (CSRF), `404`, `422`.

B.1.9 Usuários (administração)

`GET /api/usuarios`

Lista paginada de usuários (somente **admin**). Nunca inclui hash de senha.

Query string: `pagina`, `limite` (padrão 20, máx. 50).

```
curl -b cookies.txt http://localhost:3000/api/usuarios
```

```
{
  "dados": [ { "id": 1, "nome": "Administrador", "email": "admin@cuphead.local", "perfil": "admin", "criadoEm":
  ↪  "..." } ],
  "paginacao": { "pagina": 1, "limite": 20, "total": 1, "totalPaginas": 1 }
}
```

Status: `200`, `401`, `403`, `422`.

`PATCH /api/usuarios/:id/perfil`

Altera o perfil de outro usuário (somente **admin**). Vale imediatamente, inclusive para sessões já abertas.

Corpo (JSON): `perfil` (`jogador` | `admin`).

```
curl -b cookies.txt -X PATCH http://localhost:3000/api/usuarios/2/perfil \
  -H "Content-Type: application/json" -H "X-CSRF-Token: <token>" -d '{"perfil":"admin"}'
```

Resposta: `{ "dados": Usuario }` atualizado.

Status: `200`, `401`, `403`, `404`, `409` (alterar o próprio perfil), `422`.

C Resultado dos testes automatizados

```
$ npm test   # node --test (unitários + integração + segurança)▶
 API - fluxos principais
  ✔ GET /api/chefes lista com paginação e filtro (público) (18.154682ms)
  ✔ GET /api/chefes?busca encontra pelo nome sem diferenciar maiúsculas (11.080548ms)
  ✔ CRUD completo de chefe pelo administrador (52.065513ms)
  ✔ fluxo de tática: jogador envia, admin aprova, guia público exibe (43.620849ms)
  ✔ progresso de conquistas é salvo por usuário (78.210327ms)
  ✔ respostas de erro seguem o formato padronizado (11.595126ms)
  ✔ JSON malformado retorna 400 e rota inexistente retorna 404 (11.600122ms)
  ✔ eventos de login e de criação de entidade são registrados no log (0.683315ms)✔
 API - fluxos principais (514.961149ms)▶
 Segurança
  ✔ [SQLi] login com payload de injeção é rejeitado e não autentica (17.899658ms)
  ✔ [SQLi] busca com payload é tratada como texto e o banco permanece íntegro (12.609829ms)
  ✔ [SQLi] parâmetro de rota "1 OR 1=1" é recusado pela validação (8.32589ms)
```

```
 ✔ [XSS] conteúdo com <script> é devolvido apenas como JSON, com CSP e nosniff (31.94119ms)
 ✔ [CSRF] requisição sem token ou com token errado é bloqueada (403) (20.896999ms)
 ✔ [Acesso sem login] rotas protegidas retornam 401 (37.912708ms)
 ✔ [Perfil insuficiente] jogador não acessa rotas de administrador (403) (33.715416ms)
 ✔ [Sessão inválida] cookie reutilizado após logout não dá acesso (401) (47.41567ms)
 ✔ [Mass assignment] cadastro com "perfil": "admin" cria apenas jogador (17.089049ms)
 ✔ [Senhas] banco armazena apenas hash bcrypt, nunca a senha (4.58528ms)
 ✔ [Respostas] nenhuma resposta da API expõe senha_hash (25.585884ms)✔
Segurança (493.174094ms)▶
Segurança - rate limiting do login
 ✔ bloqueia com 429 após 3 tentativas erradas (59.806198ms)✔
Segurança - rate limiting do login (155.740619ms)▶
Tratamento de erros - detalhes técnicos só em desenvolvimento
 ✔ em produção o erro interno não expõe stack trace (11.463629ms)
 ✔ em desenvolvimento o erro inclui o bloco "tecnico" (14.314657ms)✔
Tratamento de erros - detalhes técnicos só em desenvolvimento (26.167757ms)▶
AuthService
 ✔ registra usuário com perfil jogador e guarda apenas o hash da senha (3.491644ms)
 ✔ recusa e-mail já cadastrado com 409 (1.348269ms)
 ✔ autentica com credenciais corretas e registra login_sucesso (0.701951ms)
 ✔ recusa senha incorreta com 401 genérico e registra login_falha (0.650469ms)
 ✔ e-mail inexistente recebe a mesma resposta e ainda executa a comparação de hash (0.87213ms)
 ✔ rejeita login com tipos inválidos antes de consultar o banco (0.404017ms)✔
AuthService (9.416748ms)▶
ChefeService
 ✔ cria chefe válido, registra o autor e loga a criação (4.099917ms)
 ✔ recusa ilha inexistente com 422 (1.085191ms)
 ✔ recusa nome duplicado com 409 (0.443209ms)
 ✔ recusa estratégia curta e dificuldade fora de 1-5 (0.35281ms)
 ✔ retorna 404 para chefe inexistente (0.471539ms)
 ✔ calcula a paginação e repassa filtros ao repositório (0.599755ms)
 ✔ escapa curingas do LIKE na busca (0.276911ms)✔
ChefeService (9.137571ms)▶
Matriz de permissões (RBAC)
 ✔ visitante só tem permissões de leitura pública (2.464415ms)
 ✔ jogador pode sugerir táticas e marcar conquistas, mas não gerenciar chefes (0.393557ms)
 ✔ administrador tem todas as permissões (1.870618ms)
 ✔ nega ações desconhecidas e perfis inexistentes (0.236201ms)
 ✔ trata ausência de usuário como visitante (0.187773ms)✔
Matriz de permissões (RBAC) (6.78829ms)▶
TaticaService
 ✔ cria tática como pendente vinculada ao usuário (2.68397ms)
 ✔ recusa tática para chefe inexistente (1.282169ms)
 ✔ autor edita a própria tática pendente (0.880764ms)
 ✔ impede que outro jogador edite ou exclua a tática (403) (0.83156ms)
 ✔ impede que o autor altere tática já moderada (409) (0.381519ms)
 ✔ administrador exclui qualquer tática, mesmo aprovada (2.987142ms)
 ✔ moderação aceita apenas "aprovada" ou "rejeitada" (0.849919ms)✔
TaticaService (12.04258ms)▶
Validação e sanitização de entradas
 ✔ sanitiza texto: remove caracteres de controle e espaços repetidos (1.401205ms)
 ✔ preserva quebras de linha em campos multilinha (0.368559ms)
 ✔ aceita um cadastro válido e normaliza o e-mail (1.980675ms)
 ✔ descarta campos fora do esquema (mass assignment) (0.315215ms)
 ✔ rejeita tipos e tamanhos inválidos com erros por campo (0.441365ms)
 ✔ exige senha com letras e números (0.384763ms)
 ✔ converte inteiros vindos como texto e valida o intervalo (0.491373ms)
 ✔ rejeita id de rota que não é inteiro positivo (ex.: tentativa de injeção) (0.441689ms)
 ✔ aplica padrões e limites de paginação (0.491229ms)✔
Validação e sanitização de entradas (8.122026ms)
tests 56
suites 9
pass 56
fail 0
cancelled 0
skipped 0
todo 0
duration_ms 2681.574747
```

D Capturas da interface

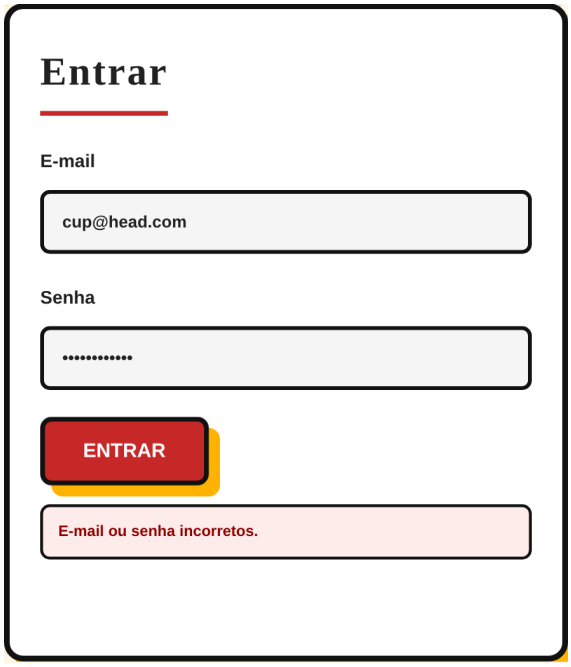


Figura 15: Login com senha incorreta

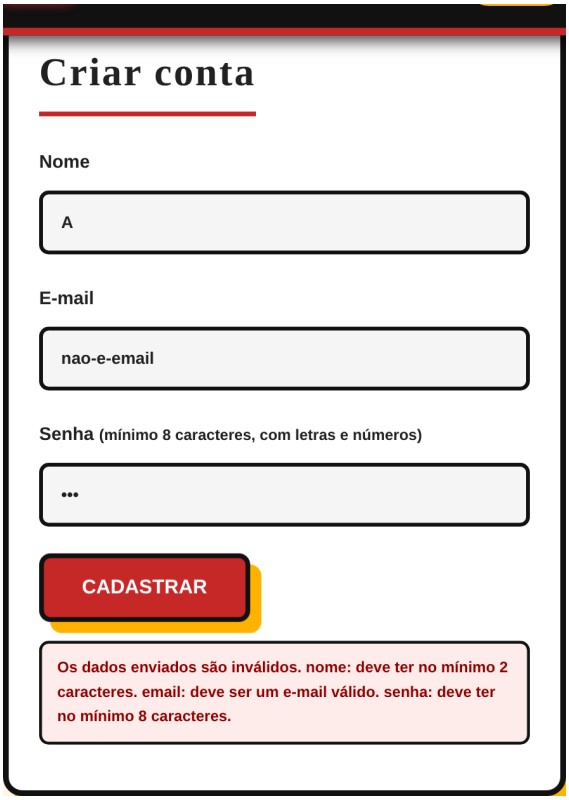


Figura 16: Cadastro com erros de validação devolvidos pela API



Figura 17: Visitante: formulário de tática bloqueado



Figura 18: Jogador: tática curta demais recusada

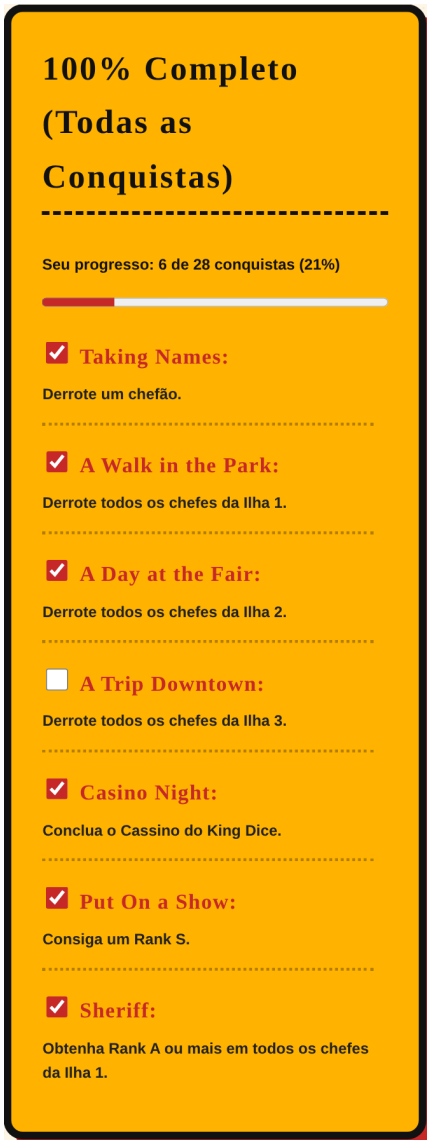


Figura 19: Progresso de conquistas salvo



Figura 20: Versão mobile (390 px)

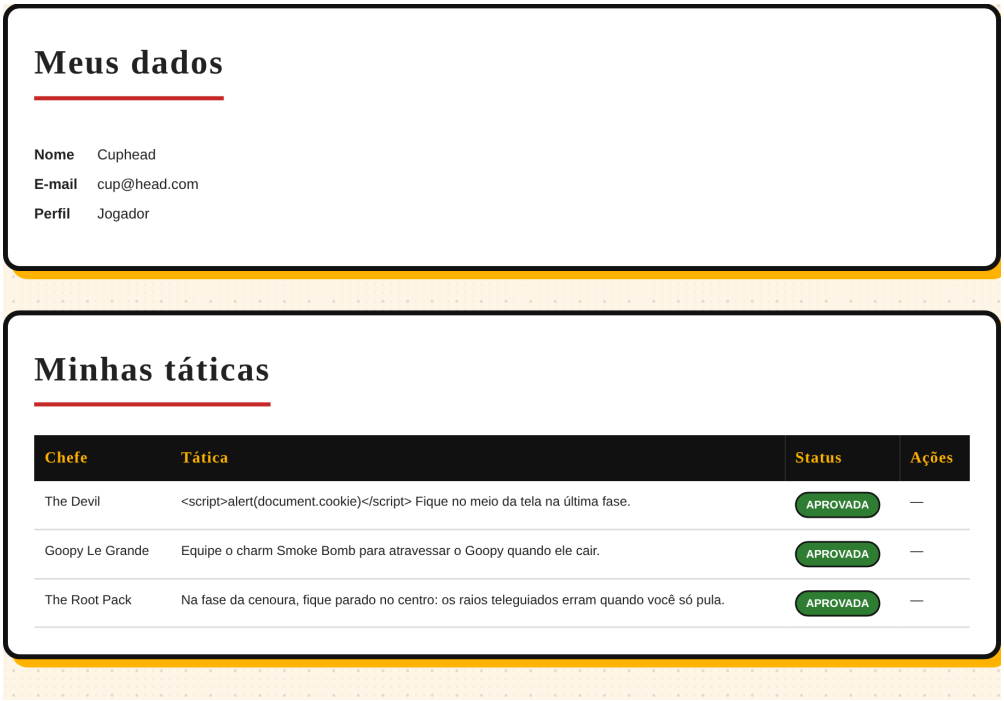


Figura 21: Página “Minha Conta” com o status das táticas enviadas

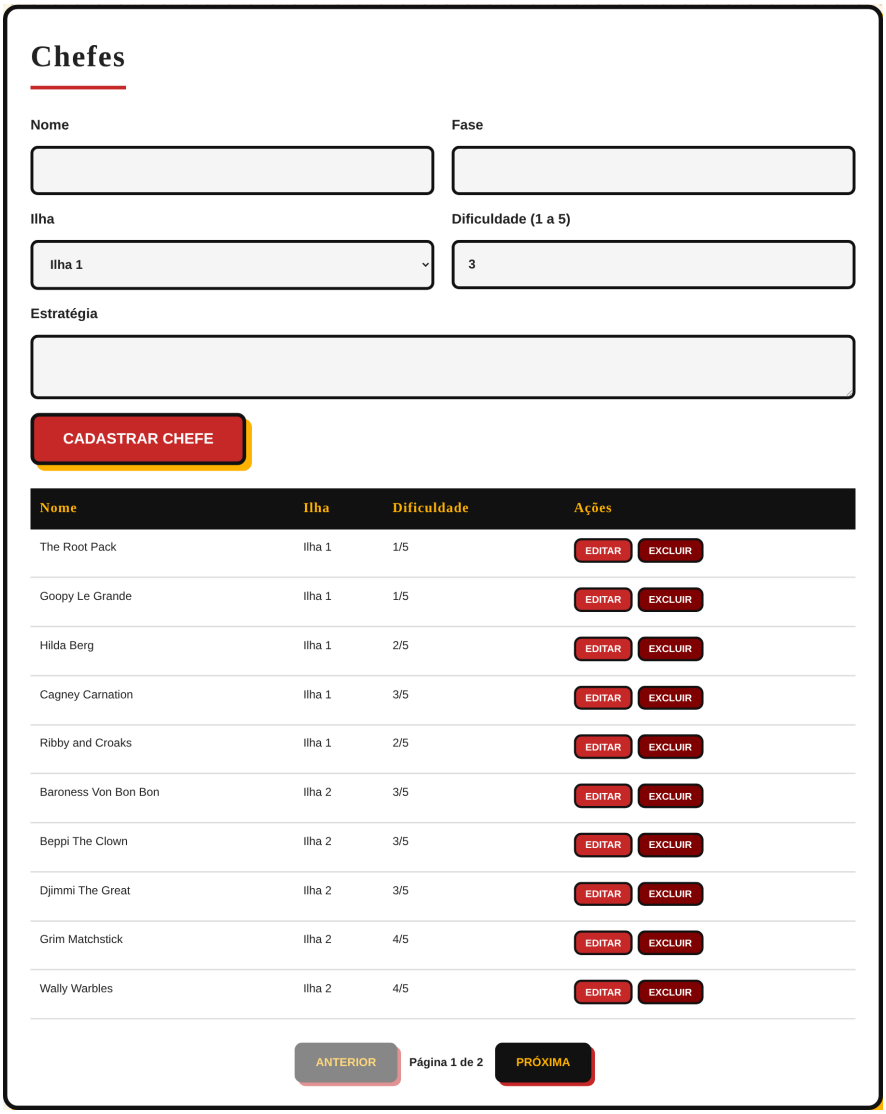


Figura 22: Painel do administrador: CRUD de chefes com paginação



Figura 23: Painel do administrador: moderação de táticas

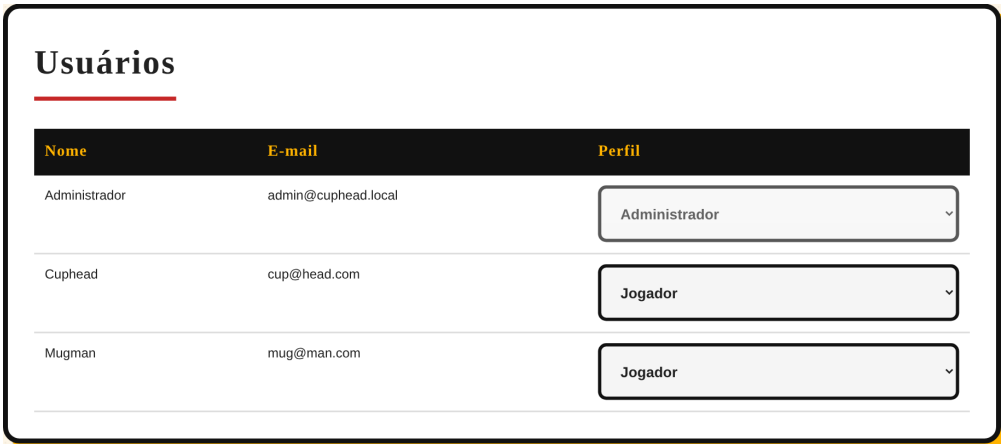


Figura 24: Painel do administrador: perfis de usuários

E Amostra dos logs

Arquivo `docs/evidencias/logs_amostra.txt` (até duas linhas de cada tipo de evento; stack traces encurtados).


```
{"timestamp":"2026-10-14T17:23:36.349Z","nivel":"info","evento":"servidor.iniciado","porta":3000,"ambiente":"development"}
{"timestamp":"2026-10-14T17:24:14.183Z","nivel":"info","evento":"entidade.criada","entidade":"usuario","id":2,"perfil":"jogador"}
{"timestamp":"2026-10-14T17:24:14.560Z","nivel":"info","evento":"entidade.criada","entidade":"usuario","id":3,"perfil":"jogador"}
{"timestamp":"2026-10-14T17:24:15.001Z","nivel":"info","evento":"auth.login_sucesso","usuarioId":2,"perfil":"jogador","ip":"127.0.0.1"}
{"timestamp":"2026-10-14T17:24:15.045Z","nivel":"info","evento":"entidade.criada","entidade":"tatica","id":1,"chefeId":1,"por":2}
{"timestamp":"2026-10-14T17:24:15.065Z","nivel":"info","evento":"entidade.criada","entidade":"tatica","id":2,"chefeId":2,"por":2}
{"timestamp":"2026-10-14T17:24:15.637Z","nivel":"info","evento":"auth.login_sucesso","usuarioId":3,"perfil":"jogador","ip":"127.0.0.1"}
{"timestamp":"2026-10-14T17:24:17.044Z","nivel":"aviso","evento":"seguranca.csrf_invalido","requisicaoId":"1f5905b1-8a5a-41e1-8cef-641f524427b2","
      metodo":"POST","rota":"/api/taticas","ip":"127.0.0.1","motivo":"cabecalho_ausente"}
{"timestamp":"2026-10-14T17:24:17.099Z","nivel":"aviso","evento":"acesso.negado","requisicaoId":"bc9766e1-86f0-46a8-a048-2263720641fe","metodo":"GET","
      rota":"/api/taticas/minhas","acao":"tatica:listar_proprias","usuarioId":null,"perfil":"visitante","ip":"127.0.0.1"}
{"timestamp":"2026-10-14T17:24:17.157Z","nivel":"aviso","evento":"acesso.negado","requisicaoId":"2b8c771a-a27b-43eb-8551-f50ea87ee228","metodo":"DELETE
      ","rota":"/api/chefes/1","acao":"chefe:excluir","usuarioId":2,"perfil":"jogador","ip":"127.0.0.1"}
{"timestamp":"2026-10-14T17:24:17.265Z","nivel":"info","evento":"auth.logout","usuarioId":1}
```

```
{"timestamp":"2026-10-14T17:24:17.781Z","nivel":"aviso","evento":"auth.login_falha","email":"admin@cuphead.local","ip":"127.0.0.1","motivo":"
       senha_incorreta"}
{"timestamp":"2026-10-14T17:24:18.124Z","nivel":"aviso","evento":"auth.login_falha","email":"admin@cuphead.local","ip":"127.0.0.1","motivo":"
       senha_incorreta"}
{"timestamp":"2026-10-14T17:24:18.923Z","nivel":"aviso","evento":"seguranca.rate_limit","tipo":"login","ip":"127.0.0.1","rota":"/api/auth/login","
       requisicaoId":"0ec3a0a8-1267-4039-8fc1-41e3bbd42ac1"}
{"timestamp":"2026-10-14T17:24:18.939Z","nivel":"aviso","evento":"seguranca.rate_limit","tipo":"login","ip":"127.0.0.1","rota":"/api/auth/login","
       requisicaoId":"e25b89bf-23bc-4145-9c03-2312984626d5"}
{"timestamp":"2026-10-14T17:24:28.747Z","nivel":"info","evento":"servidor.encerrando","sinal":"SIGTERM"}
{"timestamp":"2026-10-14T17:24:29.519Z","nivel":"info","evento":"servidor.iniciado","porta":3000,"ambiente":"development"}
{"timestamp":"2026-10-14T17:24:32.592Z","nivel":"aviso","evento":"auth.login_falha","email":"admin@cuphead.local'--","ip":"127.0.0.1","motivo":"
       usuario_inexistente"}
{"timestamp":"2026-10-14T17:24:32.793Z","nivel":"aviso","evento":"seguranca.csrf_invalido","requisicaoId":"697da569-cca0-4284-8372-e91e84d0079e","
       metodo":"POST","rota":"/api/taticas","ip":"127.0.0.1","motivo":"cabecalho_ausente"}
{"timestamp":"2026-10-14T17:24:33.016Z","nivel":"info","evento":"auth.logout","usuarioId":1}
{"timestamp":"2026-10-14T17:24:40.057Z","nivel":"info","evento":"servidor.encerrando","sinal":"SIGTERM"}
{"timestamp":"2026-10-14T17:24:43.917Z","nivel":"aviso","evento":"auth.login_falha","email":"admin@cuphead.local'--","ip":"127.0.0.1","motivo":"
       usuario_inexistente"}
{"timestamp":"2026-10-14T17:25:13.042Z","nivel":"info","evento":"entidade.atualizada","entidade":"tatica","id":1,"status":"aprovada","por":1}
{"timestamp":"2026-10-14T17:25:13.062Z","nivel":"info","evento":"entidade.atualizada","entidade":"tatica","id":2,"status":"aprovada","por":1}
{"timestamp":"2026-10-14T17:27:56.882Z","nivel":"erro","evento":"erro.interno","requisicaoId":"b9c70b6a-5ccb-4876-bfc8-3ecd5bfe2027","metodo":"GET","
       rota":"/api/saude","usuarioId":null,"mensagem":"connect ECONNREFUSED 127.0.0.1:5432","stack":"Error: connect ECONNREFUSED 127.0.0.1:5432 (...)"}
```


F Histórico Git

Saída de `git log --oneline --graph --all --decorate`. Fluxo: branches `feature/*`, `test/*` e `docs/*` integradas em `develop` com `merge --no-ff`; `develop` integrada em `main`; tags `v1.0` (Trabalho I) e `v2.0` (Trabalho II). Mensagens no padrão *Conventional Commits*.

```
*-.   af53215 (refs/stash) WIP on docs/relatorio: 0853e19 merge: docs/documentacao em develop
|\ \
| | * 00ae889 untracked files on docs/relatorio: 0853e19 merge: docs/documentacao em develop
| * 9afd585 index on docs/relatorio: 0853e19 merge: docs/documentacao em develop
|/
*   16ebb42 merge: docs/documentacao em develop
|\
| * 72b0636 docs: adiciona evidências de testes, logs, desempenho e checklist
| * b756feb fix(frontend): ajusta cabeçalho no celular e colunas de ações das tabelas
| * b711338 docs: documenta API, arquitetura e instruções de execução
| * 895be3a fix(db): reset das migrações remove apenas objetos do projeto
|/
*   4e14743 merge: test/testes-automatizados em develop
|\
| * 0f257a5 test: adiciona testes unitários, de integração e de segurança
|/
*   2d34bc6 merge: feature/frontend-integracao em develop
|\
| * 0e2087e feat(frontend): integra a interface do T1 com a API
|/
*   eaa1d07 merge: feature/taticas-conquistas em develop
|\
| * 9ef77d8 feat(taticas): persiste táticas da comunidade e progresso de conquistas
|/
*   bdebbef merge: feature/api-chefes em develop
|\
| * 0b874f3 feat(chefes): adiciona CRUD de chefes com paginação, filtros e perfis
|/
*   5309573 merge: feature/autenticacao em develop
|\
| * ad93d38 feat(auth): implementa cadastro, login e logout com sessão segura
|/
*   dd4c76e merge: feature/banco-de-dados em develop
|\
| * 9695b92 feat(db): cria modelo de dados com migrações e script SQL completo
|/
*   69c5515 merge: feature/estrutura-backend em develop
|\
| * 8d690b8 feat(backend): cria estrutura base da API com Express
|/
* a1dabf3 chore: reorganiza o projeto do T1 na pasta frontend/
```

```
* 4341785 Formatação do README.md
| *   e5db18a (HEAD -> docs/relatorio, develop) merge: docs/documentacao em develop
| |\
| | * fdd2e1e (docs/documentacao) docs: adiciona evidências de testes, logs, desempenho e checklist
| | * a53e38c fix(frontend): ajusta cabeçalho no celular e colunas de ações das tabelas
| | * 57f3e02 docs: documenta API, arquitetura e instruções de execução
| | * 2358233 fix(db): reset das migrações remove apenas objetos do projeto
| |/
| *   df599de merge: test/testes-automatizados em develop
| |\
| | * 05580e3 (test/testes-automatizados) test: adiciona testes unitários, de integração e de segurança
| |/
| *   677cd9d merge: feature/frontend-integracao em develop
| |\
| | * 06887a9 (feature/frontend-integracao) feat(frontend): integra a interface do T1 com a API
| |/
| *   5f7fd02 merge: feature/taticas-conquistas em develop
| |\
| | * ec394c8 (feature/taticas-conquistas) feat(taticas): persiste táticas da comunidade e progresso de
      conquistas
| |/
| *   0b6f037 merge: feature/api-chefes em develop
| |\
| | * e11f03d (feature/api-chefes) feat(chefes): adiciona CRUD de chefes com paginação, filtros e perfis
| |/
| *   3b46287 merge: feature/autenticacao em develop
| |\
| | * 9f99075 (feature/autenticacao) feat(auth): implementa cadastro, login e logout com sessão segura
| |/
| *   4b284ca merge: feature/banco-de-dados em develop
| |\
| | * 6c47211 (feature/banco-de-dados) feat(db): cria modelo de dados com migrações e script SQL completo
| |/
| *   3f4901c merge: feature/estrutura-backend em develop
| |\
| | * b76af74 (feature/estrutura-backend) feat(backend): cria estrutura base da API com Express
| |/
| * 0bcde00 chore: reorganiza o projeto do T1 na pasta frontend/
| * 8887065 (tag: v1.0, origin/main, origin/HEAD, main) Formatação do README.md
|/
* 0533055 Adição de interatividade tátil nos botões e efeito de rotação nos cards
* 993580c Implementação de interatividade CSS e filtros visuais na galeria
* d3cf033 Remoção de atributo obsoleta (allowtransparency) no iframe para validação W3C
* e97dbd7 Melhoria de performance com lazy loading no iframe e adição de meta description para SEO
* f6c5628 Correção de contraste no título da seção de formulário
* 3fbfd70 Estrutura inicial do HTML e CSS do guia Cuphead
```

G Checklist de requisitos

Requisito	Status	Evidência
1. Separação interface / lógica / dados	✓	Seção 4
1. Arquitetura documentada	✓	Seção 4, `docs/ARQUITETURA.md`
2. Entidade principal com CRUD	✓	Chefes; seções 5 e 8
2. Consultas parametrizadas	✓	`src/repositories/`
2. PK, FK, UNIQUE, NOT NULL	✓	Seção 5, Anexo A
2. Script de criação do banco	✓	`script_completo.sql`

Requisito	Status	Evidência
3. Sessão segura	✓	Seção 6
3. Hash bcrypt	✓	Seção 6; teste `[Senhas]`
3. Logout funcional	✓	Teste 6 (seção 10)
3. Proteção de rotas	✓	Testes 4 e 5
4. Dois ou mais perfis	✓	Visitante, jogador, admin
4. Matriz de permissões	✓	Tabela 5
5. Validação de tipo e tamanho	✓	Seção 7.1
5. Sanitização	✓	`sanitizarTexto()`
5. Erros estruturados	✓	Seções 7.1 e 7.2
6. SQL Injection	✓	Testes 1 e 1b
6. XSS refletido	✓	Teste 2, Figura 8
6. CSRF	✓	Teste 3
6. Acesso não autorizado	✓	Testes 4, 5 e 6
7. Erros padronizados; detalhes só em dev	✓	Figura 4; testes automatizados
8. Documentação de todos os endpoints	✓	Anexo B (25 endpoints)
9. ≥ 5 testes, válidos e inválidos	✓	56 testes; Anexo C
10. Logs com timestamp (4 eventos)	✓	Seção 11; Anexo E
Opcional: `.env`	✓	`backend/.env.example`
Opcional: migrações	✓	`database/migrations/`
Opcional: paginação	✓	`?pagina=&limite=`
Opcional: rate limiting	✓	Teste 7
Opcional: testes de integração	✓	22 testes em `tests/integracao/`
Git com branches e tags	✓	Anexo F
Sem credenciais no repositório	✓	Só `.env.example` versionado